Basic Understanding of Neural Networks

1. *What is the fundamental structure of an Artificial Neural Network (ANN)?*

ANN is a foundational machine learning model built around the mathematical concept of a neuron. The fundamental structure of an Artificial Neural Network (ANN) is built around a mathematical model of a neuron, which consists of the following components working together in sequence:

- **Inputs (x):** The raw data or signals fed into the network, denoted as $x_1, x_2, \dots, x_n$.
- **Weights (ω):** Learnable parameters assigned to each specific input line that determine the importance or influence of that input.
- **Bias (b or ω_0):** An additional learnable parameter added to the node, acting similarly to an intercept in a linear equation.
- **Summation (Σ):** The neuron calculates a weighted sum of all inputs plus the bias. This is mathematically expressed as
- **Activation Function (φ or σ):** A mathematical function applied to the weighted sum. It acts as a **"gatekeeper"** that decides whether or not to pass a signal forward, and how strongly. Common examples include ReLU, Logistic (sigmoid), Tanh, Identity, and Binary step functions.
- **Output (y):** The final prediction or result produced by the neuron after the activation function is applied, represented by the equation.

When scaled up into a deeper Feedforward Neural Network configuration, such as a MultiLayer Perceptron (MLP), these individual neurons are organized into distinct stages: an **Input layer**, one or more interconnected **Hidden layers**, and a final **Output layer**.

2. *What are the key steps involved in training a neural network from scratch?*

Training a neural network from scratch involves a specific five-step sequence. The first step is foundational, while the remaining four steps form an iterative "Training Loop" that allows the model to learn and improve over time. Here are the key steps involved:

1. Build the Neural Network Before any learning can happen, you must design the initial framework of the model. This involves three main decisions:

- **Structure (Architecture):** Defining the layers, such as input layers, hidden layers, and the output layer, as well as the activation functions that will process the signals.
- **Loss Function:** Selecting a mathematical formula (such as Mean Absolute Error (MAE), Mean Squared Error (MSE), or R^2) that will be used to measure how wrong the network's guesses are.
- **Optimization Approach:** Determining the method (like Batch or Stochastic Gradient Descent) the network will use to minimize its errors.

2. Forward Pass (Prediction) Once the network is built, the actual data is fed into it. The forward pass is the **process of predicting the output value given the input**. The input data (x) flows forward through the network's weights and activation functions to generate a

final prediction ($\hat{y}$). During this step, the network is simply making its best guess based on its current, unoptimized parameters.

3. FeedForward Evaluation In this step, the network assesses its own performance. It takes the prediction generated in the Forward Pass and compares it against the actual, correct target value. It uses the previously defined **Loss Function** to calculate the exact amount of error.

4. Backward Pass (Gradient Calculation) To figure out how to fix the error, the network must determine which weights are responsible for the mistake. It does this by calculating the gradient of the loss with respect to each weight.

- **The Chain Rule:** Because neural networks are nested functions, the system uses the calculus chain rule () to trace the derivative of the error backward through every layer.
- This calculation provides both the **direction and magnitude** to adjust each weight (w)so that the model can reduce its loss in the next iteration.

5. Back Propagation (Update Weights) This is the step where the actual "learning" occurs. Using the gradients calculated in the previous step, the network updates its parameters to minimize the loss.

- **Gradient Descent Formula:** The weights are updated using the formula:
- In this equation, the network subtracts the gradient (∇J) scaled by a **learning rate (α)** from the old weight (w_{old}) to create a new, optimized weight (w_{new}).

Steps 2 through 5 are **repeated iteratively** (often thousands of times over many epochs) until the network finds the parameter values that minimize the loss function and achieves convergence.

3. *How do we evaluate the performance and generalization of a model?*

Evaluating the performance and generalization of a model depends heavily on the specific architecture being trained and the tasks it is designed to perform. The evaluation process ranges from calculating mathematical loss during training to utilizing human judges for complex generative outputs.

Here is a breakdown of how model performance is evaluated based on the course materials:

1. Foundational Evaluation During Training
At the most basic level of training an Artificial Neural Network, performance is assessed during the **FeedForward Evaluation** step of the training loop.

- **Loss Functions:** The network uses a loss function to mathematically measure the error between its prediction ($\hat{y}$) and the actual target value (y).

- **Common Metrics:** Typical loss functions used to quantify this error include **Mean Absolute Error (MAE)**, **Mean Squared Error (MSE)**, **Root Mean Squared Error (RMSE)**, and R^2. This calculation is crucial because it provides the gradient needed to update the model's weights and improve its performance.

2. Task-Specific Statistical Metrics
Once a model is trained, different quantitative metrics are used to evaluate its performance depending on the specific NLP or predictive task:

- **Classification:** Evaluated using **Accuracy** and **F1 score**. For specialized classification tasks (like the Life2Vec model predicting life outcomes), the **Matthews Correlation Coefficient (MCC)** is used. MCC considers all four outcomes of a confusion matrix (True Positives, True Negatives, False Positives, False Negatives) and provides a score from -1 (total disagreement) to +1 (perfect prediction).

- **Text Generation:** Evaluated using **Perplexity** (which measures how well a model predicts text, where lower is better) and **BLEU score**.

- **Summarization:** Evaluated using **ROUGE score** (which assesses the overlap between the model's output and human-generated text).

- **Translation:** Evaluated using **BLEU score** and **METEOR**, which improves upon BLEU by factoring in synonyms and word order.

- **Question-Answering:** Uses **Exact Match (EM)** and **F1 score** for extractive QA, while relying on **BLEU / ROUGE** for generative QA tasks.

3. Evaluating Large Language Models (LLMs)
Because Large Language Models generate complex, open-ended text, evaluating their performance and generalization requires broader philosophies. The sources outline four primary ways to "grade" an LLM:

- **Deterministic / Statistical (NLP Metrics):** This philosophy dictates that you evaluate a model by measuring the word overlap or semantic embedding similarity between its output and a known "Ground Truth" or reference answer. This uses tools like BLEU, ROUGE, Exact Match, and **BERTScore** (which calculates semantic similarity using contextual embeddings rather than just exact word matches).

- **LLM-as-a-Judge (Neural Evaluation):** This method uses a more capable, advanced model acting as a "Judge" to score a candidate model's outputs based on a specific rubric and context. Common tools for this include DeepEval, Ragas, G-Eval, and Prometheus.

- **Human-in-the-Loop (Human Preference Evaluation):** This approach relies on human annotators utilizing comparison UIs (such as Chatbot Arena, Argilla, or Labelbox). The core philosophy here is that "humans are the final arbiters of helpfulness, safety, and nuance".

- **Benchmark Suites (Standardized Benchmarking):** To test how well a model generalizes and to allow for head-to-head comparisons across different models, researchers use curated test sets. These suites (such as MMLU-Pro, GPQA, HumanEval, and SWE-bench) evaluate models on standardized coding, math, and multiple-choice questions.

4. *Which optimization methods are used to find the ideal values for weights and biases?*

To find the ideal values for parameters like weights and biases, neural networks rely on an iterative optimization process called **Gradient Descent**. This process uses the gradient of the cost or loss function to adjust parameters in the direction that reduces the error, following the formula:

The sources detail three main types of gradient descent methods used to update these parameters:

- **Batch Gradient Descent:** This method computes the gradient of the cost function using the **entire training dataset**. While it is computationally efficient for datasets that can fit entirely into memory (benefiting from vectorized operations), it can be very slow when processing extremely large datasets.

- **Stochastic Gradient Descent (SGD):** Instead of using the whole dataset, SGD computes the gradient and updates the parameters for **one training example at a time**. This allows the network to handle massive datasets since only a single example needs to be stored in memory at a time. However, this approach makes the path to the minimum error "noisy" and leads to less accurate convergence compared to the Batch method.

- **Mini-Batch Gradient Descent:** This method strikes a balance by updating parameters using a **subset (or mini-batch) of the training data**. It offers faster computation than Batch Gradient Descent because it doesn't process the entire dataset before making an update. However, it introduces a new hyperparameter—the mini-batch size—which can be challenging to tune perfectly.

5. *How do Batch Gradient Descent and Stochastic Gradient Descent (SGD) differ in practice?*

Batch Gradient Descent and **Stochastic Gradient Descent (SGD)** differ primarily in how much data they process to update model parameters, which significantly impacts their speed, memory usage, and how they converge:

- **Data Processed per Update: Batch Gradient Descent computes the gradient of the cost function using the entire training dataset** before making a single parameter update. In contrast, **SGD updates the parameters for each individual training example one at a time**.

- **Memory Efficiency and Scalability:** Because it processes everything at once, Batch Gradient Descent can benefit from vectorized operations, making it **computationally efficient only when the entire dataset can fit into memory**. However, it becomes **very slow when dealing with large datasets**. On the other hand, **SGD can handle massive datasets easily** because it only requires keeping a single training example in memory at any given time.

- **Convergence Behavior:** Because SGD updates parameters based on single examples rather than the whole dataset, its **path to the minimum error is "noisy"** compared to Batch Gradient Descent. This volatility ultimately leads to **less accurate convergence** than the Batch method.

6. *What is a **loss function**, and how does it guide the learning process?*

A **loss function** (or cost function) is a mathematical formula used to evaluate the performance of a neural network by measuring the error between its prediction ($\hat{y}$) and the actual target value (y). Common examples of loss functions include Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and R^2.

In the context of the training process, the loss function acts as the "compass" that guides the network on how to improve. It does this through the following mechanisms:

- **Measuring the Error:** During the FeedForward Evaluation step, the loss function calculates exactly how wrong the model's current predictions are based on its current weights.

- **Providing a Landscape for Optimization:** You can visualize the loss function as a curved landscape where the goal is to reach the lowest possible point (the minimum error).

- **Calculating the Gradient:** During the Backward Pass, the network calculates the derivative (the rate of change or slope) of the loss function with respect to each weight. This calculation tells the network whether the slope of the error is going uphill or downhill.

- **Directing Weight Updates:** Most importantly, the gradient of the loss function provides both the **direction and magnitude** needed to adjust the weights to minimize the loss. In the Back Propagation step, the network uses the gradient descent formula to subtract a portion of the loss function's gradient from the old weights, iteratively stepping closer to the optimal parameters.

Essentially, without a loss function, the network would have no way of knowing if it is making good or bad predictions, and no mathematical baseline to calculate how to adjust its parameters to learn.

7. *What is the distinction between **parameters** and **hyperparameters** in a neural network?*

The difference between **parameters** and **hyperparameters** is who determines them.

- **Parameters** are internal values the neural network **learns automatically during training**, mainly **weights and biases**. They are updated using **backpropagation and gradient descent** to minimize the loss function. Modern models can have **millions or trillions of parameters** (e.g., BERT or GPT-4).

- **Hyperparameters**, these are external settings that **you must configure manually** before training or text generation begins. They control *how* the model learns or behaves. Examples include the **learning rate**, **mini-batch size**, **temperature**, and **top-k/top-p** sampling.

In short:

- **Parameters = knowledge learned from data.**
- **Hyperparameters = rules set by the user that control the learning or generation process.**

8. *How does **Gradient Descent** mathematically minimize the loss function?*

Gradient descent mathematically minimizes the loss function through an iterative process that uses calculus to find the "downhill" slope of the error and continuously adjusts parameters until it reaches the lowest possible point.

Here is the step-by-step mathematical breakdown based on the course materials:

1. Finding the Gradient (The Slope) The core mathematical tool in gradient descent is the derivative, which measures the rate at which a function changes at a specific point. During the Backward Pass step, the network calculates the derivative of the loss function with respect to each specific weight, typically using the **chain rule** () to trace the error backwards through the layers.

- This calculation reveals the slope of the error curve: a positive slope means it is going uphill, a negative slope means it is going downhill, and zero means it is perfectly flat.

- Most importantly, this gradient provides the exact **direction and magnitude** needed to adjust the weight () to minimize the loss.

2. The Gradient Descent Formula Once the gradient is calculated, the actual minimization happens during the Back Propagation step using this specific mathematical formula:

3. Breakdown of the Equation

- w_{new}: The updated parameter vector.
- w_{old}: The current parameter vector.

- α: The **learning rate**, which acts as a multiplier determining the "step size" the algorithm takes in each iteration.

- $\nabla J(w_{old})$: The **gradient** of the cost or loss function () with respect to the parameters at their current values.

4. The Mechanism of Minimization The mathematical key to minimizing the error is the **subtraction** sign in the formula. By subtracting the gradient (scaled by the learning rate) from the old parameter, the algorithm forces the weight to adjust in the opposite direction of the slope. This guarantees that the parameter is continually adjusted "in the direction that reduces the cost function".

The network applies this formula **iteratively** over many cycles until it reaches convergence, successfully finding the specific parameter values that minimize the cost function.

9. *What does **Mean Squared Error (MSE)** represent, and when is it the appropriate metric to use?*

Mean Squared Error (MSE) is a **loss function** that mathematically represents the average of the squared differences between the model's predicted values ($\hat{y}$) and the actual, correct values (y_i). Based on the provided slides, its formula is defined as:

Regarding when it is appropriate to use, the materials specifically demonstrate MSE being utilized as the loss function for a **linear regression model**. During the backward pass of a linear regression model, the gradient of the MSE is calculated to determine the exact direction and magnitude needed to adjust the weights to minimize the model's error.

Because MSE squares the error terms, it heavily penalizes larger mistakes. Therefore, it is generally the appropriate metric to use in regression tasks where you want to ensure the model makes as few large errors as possible.

10. *How does the **backpropagation** algorithm calculate gradients across layers?*

The backpropagation algorithm relies on a foundational concept from calculus known as the **Chain Rule** to calculate gradients backward through the layers of a neural network.

Because a neural network processes data through a sequence of layers, it mathematically functions as a series of nested operations where one function is composed of another function. The chain rule allows the algorithm to systematically unpack these nested layers to determine exactly how much a specific weight contributed to the final error.

The sources provide a concrete step-by-step example of how this works during the backward pass of a linear regression model using a Mean Squared Error (MSE) loss function:

1. **Find the derivative of the error:** First, the algorithm calculates how the loss changes with respect to the prediction ($\hat{y}$).

2. **Find the derivative of the prediction:** Next, it calculates how the prediction changes with respect to the weight (w_i).

3. **Apply the Chain Rule:** It multiplies these intermediate derivatives together to trace the error back to the weight.

4. **Calculate the final gradient:** Finally, it averages this calculation over all data points to get the full gradient.

The resulting gradient provides the network with the precise **direction and magnitude** required to adjust that specific weight to minimize the loss during gradient descent.

Challenges Across Deep Layers While the chain rule is effective, backpropagating gradients across networks with very deep layers (such as typical Recurrent Neural Networks) can create significant mathematical problems. Because the chain rule relies on repeated multiplication across these layers, it often leads to:

- **Vanishing Gradients:** The gradients shrink exponentially as they move backward, becoming so small that the earlier weights stop updating, leading to a state of "No Learning".

- **Exploding Gradients:** The gradients blow up exponentially, creating massive, erratic updates that make the model completely unstable.

11. *How does the **learning rate** influence the convergence of Gradient Descent?*

Based on the provided course materials, the learning rate (denoted as α) influences the convergence of Gradient Descent by determining the **"step size"** the algorithm takes during each iteration of the training process.

During the Back Propagation step, the network updates its parameters to minimize the loss using the gradient descent formula.

In this equation, the learning rate (α) acts as a multiplier for the calculated gradient. By controlling how large of a mathematical step the algorithm takes in the direction that reduces the cost function, it directly drives how the network iteratively moves toward **convergence**—the point where it successfully finds the parameter values that minimize the cost function.

Information Outside of Your Sources: *While the slides define the learning rate as the step size toward convergence, they do not explicitly detail the consequences of setting it incorrectly. For your exam, it is helpful to know the standard machine learning principle:*

- *If the learning rate is **too high**, the step size is too large, and the algorithm may overshoot the lowest point of the error curve, causing it to bounce around or completely fail to converge (diverge).*
- *If the learning rate is **too low**, the step size is tiny, meaning the algorithm will take an extremely long time to reach convergence, making the training process inefficient.*

12. *Why is it standard practice to split data into training, validation, and testing sets?*

As noted in our previous discussion, the provided course slides do not explicitly explain *why* it is standard practice to split data into training, validation, and testing sets.

The sources only mention these concepts briefly in passing:

- A flowchart for fine-tuning a SetFit model includes a specific step for the "Training and validation of the multilingual classification model".

- When discussing how to evaluate Large Language Models, the slides mention using "Curated Test Sets" to benchmark and compare different models.

- The first lecture poses the study question, "What strategies can be employed to mitigate the issue of overfitting in a complex neural network?".

Information Outside of Your Sources: Since this is for your exam, here is the standard machine learning explanation for this practice, which directly answers the study question about mitigating overfitting:

The primary reason for splitting data is to ensure the model actually learns the underlying patterns and **generalizes** well to new data, rather than just memorizing the data it was trained on (which is known as **overfitting**).

- **Training Set:** This is the data used by the model during the training loop to calculate gradients and update its weights and biases.
- **Validation Set:** This is a separate chunk of data used *during* the training process to tune hyperparameters and monitor the model's performance. If the model's error continues to drop on the training set but starts increasing on the validation set, it is a clear signal that the model is overfitting and memorizing the training data.
- **Testing Set:** This data is kept completely hidden from the model until the very end of the process. It is used to provide an unbiased, final evaluation of how the model performs on completely unseen, real-world data.

Recurrent Neural Networks (RNNs) and LSTMs

1. *How do RNNs leverage sequential data for tasks like time series forecasting?*

RNNs leverage sequential data by incorporating a built-in **"Working Memory"** loop into their architecture.

Unlike a standard MultiLayer Perceptron (MLP)—which treats every input independently and has no mechanism to share information across a sequence—an RNN feeds information from past steps forward. When processing a current input vector (xt), the RNN also factors in a hidden state (ht) from the previous step. This looping mechanism allows the network to learn the relationships between events and recognize common features that occur across different parts of a sequence.

Note: While your course materials do not explicitly use the phrase "time series forecasting," they discuss this exact mechanism in the broader context of "sequential processes." The slides emphasize that RNN architectures are necessary for tasks where **"the meaning of an action often depends on what comes before and after it"**. In the course, this is applied to predicting future outcomes based on chronological histories, such as a person's sequence of life events (Life2Vec) or a firm's sequence of past product innovations (Market2Vec).

To handle sequences more effectively, the sources also highlight an upgraded version of the RNN called **Long Short-Term Memory (LSTM)**. LSTMs add a dedicated **"Long-term Memory"** track alongside the working memory to help the network retain information across much longer sequence windows.

However, as we covered in our earlier discussion about the training process, relying on these sequential memory loops creates mathematical vulnerabilities during the Backward Pass. This

makes typical RNNs highly susceptible to **vanishing and exploding gradients** as the network attempts to backpropagate errors backwards through deep, sequential layers.

2. *How does the **learning rate** influence the convergence of Gradient Descent?*

Based on the provided course materials, the learning rate (denoted by the symbol α) influences the convergence of Gradient Descent by determining the **"step size"** the algorithm takes during each mathematical iteration.
During the **Back Propagation** step, the network updates its parameters in the direction that reduces the cost or loss function using the following gradient descent formula:

In this equation, the learning rate (α) acts as a multiplier for the calculated gradient ($\nabla J(w_{old})$). By controlling how large of a step the algorithm takes, it directly drives how the network iteratively moves toward **convergence**—which is the point where the algorithm successfully finds the parameter values that minimize the cost function.

Information Outside of Your Sources: *As noted in our previous conversation, while your slides define the learning rate as the step size toward convergence, they do not explicitly detail the consequences of setting it incorrectly. For your exam, you may want to independently verify the following standard machine learning principles:*

- *If the learning rate is **too high**, the step size is too large, and the algorithm may overshoot the lowest point of the error curve, causing it to bounce around or completely fail to converge (diverge).*

- *If the learning rate is **too low**, the step size is tiny, meaning the algorithm will take an extremely long time to reach convergence, making the training process highly inefficient.*

3. *What is an **LSTM** network, and how does it solve the "vanishing gradient" problem found in traditional RNNs?*

A Long Short-Term Memory (LSTM) network is an advanced type of Recurrent Neural Network (RNN) specifically designed to better handle and remember sequential data.

In traditional RNNs, the backpropagation process struggles when dealing with deeper networks or longer sequences, often resulting in the **vanishing gradient problem**. As the algorithm calculates errors backwards through the layers, the gradients can shrink exponentially. When a gradient becomes too small, it prevents the network's weights from updating, which ultimately traps the model in a state of "No Learning".

An LSTM solves this problem by upgrading the underlying architecture to regulate how information is stored and forgotten over time:

- **Dual Memory Tracks:** While a standard RNN relies solely on a continuous "Working Memory" loop, an LSTM introduces a separate, dedicated **"Long-term Memory"** track (referred to in the network as the Cell State).

- **The Gate Mechanism:** To control the flow of information into and out of this long-term memory, an LSTM utilizes a specific system of three gates: a **Forget Gate**, an **Input Gate**, and an **Output Gate**.

By concatenating the previous working memory with the current input, these gates act as filters that selectively decide exactly what information should be forgotten, what new information should be stored in the long-term memory, and what should be passed forward as the output. This architecture effectively allows the network to retain crucial information across much longer sequence windows, mitigating the extreme gradient shrinking that cripples standard RNNs.

Advanced Neural Network Concepts

1. *What are **attention mechanisms**, and how do they allow models to focus on relevant parts of an input?*

Attention mechanisms are a foundational component of modern Deep Learning architectures (specifically the Transformer model) that allow a network to weigh the importance of different words or tokens in a sequence, regardless of their position or distance from one another.

Unlike traditional Recurrent Neural Networks (RNNs) or LSTMs that process data strictly sequentially step-by-step, attention mechanisms utilize an **"infinite reference window"**. This allows the model to look across an entire input sequence simultaneously, dispensing with recurrence entirely, to compute how strongly different parts of a sentence relate to each other.

1. The Q, K, V Matrices (Scaled Dot-Product Attention) The core of the mechanism is the **Scaled Dot-Product Attention** function. To calculate attention, the model computes three representations from the input data: **Queries (Q)**, **Keys (K)**, and **Values (V)**.

- The model calculates the dot product between a Query and all Keys. This mathematically determines the "compatibility function" or relevance score between different tokens.
- It then divides this result by a scaling factor ($\sqrt{d_k}$) and applies a **softmax function** to convert these scores into a probability distribution of weights.
- Finally, these softmax weights are multiplied by the Values (V) to produce the final output.
- This entire process is expressed in the fundamental equation.

2. Contextualization via Self-Attention By allowing tokens to "look at each other" (which is known as self-attention), the mechanism solves the problem of ambiguous context. For example, the sources illustrate how the word "bank" has two completely different meanings in the phrases "The **bank** of the river" and "Money in the **bank**". When applying self-attention, the model calculates the relevance between "bank", "river", and "money" within their respective sequences. It uses these relationships to shift the original, static word embeddings into contextually rich **Self-Attention Output Vectors**.

3. Multi-Head Attention Instead of performing this calculation just once, Transformers use **Multi-Head Attention**, which consists of several attention layers running in parallel. This

allows the model to dynamically learn and weigh differently connected views and structural patterns of the input data at the exact same time.

*(Note for your exam: Depending on the architecture, the Transformer uses different variations of this mechanism. For example, **Encoder self-attention** (used in BERT) allows all tokens to look at each other bidirectionally, while **Masked Decoder self-attention** (used in GPT models) forces tokens to only look at previous tokens so the model can learn to predict the next word.)*

2. *Describe the core architecture of the **Transformer** model.*

Based on the provided course materials, the Transformer model (introduced in the landmark 2017 paper "Attention Is All You Need") revolutionized deep learning by dispensing with sequential recurrence completely and relying solely on **attention mechanisms**.
The core architecture consists of two main sections: an **Encoder** and a **Decoder**. Here is a breakdown of the structural components that make up this architecture:

1. Input Embeddings and Positional Encoding Because the Transformer does not process text sequentially (word-by-word) like an RNN, it needs a way to understand the order of the sequence.

- It takes the initial input embeddings and adds a **Positional Encoding** to them.
- These encodings use mathematical sine and cosine functions to give the model a sense of absolute and relative word positions (e.g., distinguishing between "I am a robot" and "a robot am I").

2. Multi-Head Attention Sub-layers As we discussed in the previous question, the core engine of the architecture is the **Multi-Head Attention** mechanism, which allows the model to calculate "Scaled Dot-Product Attention" across several parallel layers simultaneously. The architecture utilizes this in three different ways:

- **Encoder self-attention:** Tokens in the input sequence look at each other to gather bidirectional context.
- **Decoder self-attention (masked):** Used in the generation phase. It forces tokens to only look at previous tokens, masking future words so the model learns to predict the next word.
- **Decoder-encoder attention:** The target token in the decoder looks back at the queries, keys, and values from the encoder states to pull relevant source information.

3. Feed-Forward Networks After the attention mechanisms allow tokens to gather contextual information from one another, the data is passed through a dense Feed-Forward network. As the slides describe it, this is where the model, after taking information from other tokens, "takes a moment to think and process this information".

4. Add & Norm (Residual Connections) Throughout both the encoder and decoder, each attention and feed-forward sub-layer is wrapped in an **"Add & Norm"** block. This represents residual connections and layer normalization, which help stabilize the network and ensure gradients can flow smoothly during the backpropagation training process.

How it evolved: It is worth noting for your exam that while the original 2017 Transformer used both an Encoder and a Decoder for translation tasks, modern Large Language Models often split this architecture. For example, **BERT** utilizes only a multi-layer bidirectional

Transformer Encoder to understand text (using Masked Language Modeling), while generative models like **GPT** rely on the **Decoder** structure to predict the next word sequentially (using Causal Language Modeling).

3. *What are the different types of text embeddings (e.g., word-level vs. sentence-level)?*

Here is a brief summary of the different types of embeddings covered in the course:

- **Word/Token-Level Embeddings:** These represent individual words or sub-words mathematically. Examples include Word2Vec and the WordPiece embeddings utilized by BERT models.
- **Sentence/Sequence-Level Embeddings:** These capture the overall semantic meaning of an entire sentence or sequence. Examples include the aggregate [CLS] token in BERT, SBERT (which computes sentence embeddings to easily compare their cosine similarity), and SetFit (which uses these sentence embeddings for efficient text classification).
- **Event/Concept Embeddings (Beyond Text):** These apply the structural rules of language to chronological socioeconomic sequences. Examples include **Life2Vec** (embedding sequences of human life events to predict outcomes like mortality) and **Market2Vec** (embedding a firm's sequence of past products to predict future innovations).

4. *What is the difference between **pre-training** and **fine-tuning**?*
    - *What are the common self-supervised objectives for pre-training?*
    - *What strategies are used for task-specific fine-tuning?*

Here is a brief summary of the differences between pre-training and fine-tuning, the objectives, and the strategies:

- **The Difference:** Pre-training builds a model's foundational language understanding using massive amounts of unlabelled data, which is computationally expensive and takes hours to days. Fine-tuning takes that pre-trained model and adapts it to a specific task using a much smaller set of labelled data, which typically only takes minutes to hours.

- **Pre-Training Objectives:** During the foundational phase, models learn through self-supervised tasks like **Masked Language Modeling** (predicting randomly hidden words using context), **Next Sentence Prediction** (determining if one sentence logically follows another), or **Causal Language Modeling** (predicting the next word in a sequence).

- **Fine-Tuning Strategies:** Because updating all of a massive model's parameters (conventional fine-tuning) is highly expensive, researchers use **Parameter-Efficient Fine-Tuning (PEFT)**. PEFT methods freeze the original model and only train a tiny subset of parameters. Common PEFT strategies include additive methods like **Soft Prompts** or reparameterization methods like **LoRA (Low-Rank Adaptation)**, which drastically reduce memory requirements and training time.

5. *Contrast the utility of word embeddings with that of global sentence embeddings.*

- **Word Embeddings (Token-Level):** These represent individual words or sub-words (such as Word2Vec or WordPiece) and act as the foundational raw input data for models. However, on their own, static word embeddings cannot resolve ambiguous contexts without utilizing mechanisms like self-attention.

- **Global Sentence Embeddings (Sequence-Level):** These compress an entire text sequence into a single aggregate vector, such as BERT's special [CLS] token or SBERT. They are highly useful for macro-level tasks, such as calculating cosine similarity to compare how related documents are, and for enabling efficient few-shot text classification frameworks like SetFit.

Exploring Large Language Models (LLMs)

1. *Why is it often necessary to adapt a foundation model for specialized domains or tasks?*

Foundation models are initially pre-trained on massive, general datasets to learn broad language representations, but this general training does not prepare them for the specific nuances and objectives of specialized domains. Adapting these pre-trained models using task-specific labelled data is necessary to achieve high accuracy on specialized downstream applications, such as question-answering or text classification. Furthermore, using targeted adaptation techniques, such as Parameter-Efficient Fine-Tuning (PEFT) or prompt engineering, allows developers to efficiently tailor the model to these specific tasks while optimizing computational resource usage and drastically reducing training time.

2. Ho*w does **LoRA (Low-Rank Adaptation)** achieve "parameter-efficient" fine-tuning?*

LoRA (Low-Rank Adaptation) enables parameter-efficient fine-tuning by avoiding updates to the entire model. Instead, it **freezes the original weights** and adds small trainable matrices to Transformer layers.

Mathematically, rather than retraining the large weight matrix W, LoRA learns two smaller low-rank matrices $A$and B. The model output becomes $h = Wx + BAx$. After training, these matrices can be merged back into the original weights.

Because only the small matrices are trained, the **number of trainable parameters is drastically reduced** (e.g., a rank of 8 may be only ~0.39% of the full matrix). In practice, LoRA reduced GPT-3 fine-tuning parameters by **10,000×** and **cut GPU memory usage by about 3×**.
Developers can apply LoRA selectively:

- **Attention layers** – best for style or instruction tuning
- **FFN layers** – better for task-specific knowledge
- **Attention + FFN** – strongest performance but higher compute cost.

3. *Define **synthetic data** and identify a major risk (such as model collapse) associated with using it.*

- **Synthetic Data:** This is data that is artificially generated by a computer algorithm or an AI model, rather than being gathered from real-world events or human annotations. In the current LLM ecosystem, developers often use highly capable models (like GPT-4) to generate vast amounts of synthetic text, code, or images to pre-train or fine-tune smaller, newer models.

- **The Risk of Model Collapse:** Model collapse is a degenerative mathematical process that happens when models are continuously trained on synthetic data generated by *other* models, rather than human-generated data. Over multiple generations of this feedback loop, the models begin to lose information about the "tails" of the original data distribution (meaning they forget rare edge cases and nuances). Eventually, the models converge on a highly homogeneous, repetitive state, producing degraded, low-quality outputs. In short: training AI exclusively on AI-generated data causes the models to "forget" the true complexity of human language and real-world data.

4. *What are the primary benchmarks and metrics used to evaluate LLMs today?*

Evaluating **Large Language Models (LLMs)** requires multiple methods because their outputs are open-ended. Four main frameworks are used:

1. **Deterministic / Statistical Metrics** – Compare model output with a reference answer. Common metrics include **Perplexity**, **BLEU**, **METEOR**, **ROUGE**, **Exact Match / F1**, and **BERTScore**.

2. **Benchmark Suites** – Standardized test sets used to compare models across tasks like reasoning, math, coding, or knowledge. Examples: **MMLU-Pro, GPQA, HumanEval, SWE-bench**.

3. **LLM-as-a-Judge** – A stronger LLM evaluates another model's output using a rubric. Tools include **DeepEval, Ragas, G-Eval, and Prometheus**.

4. **Human-in-the-Loop Evaluation** – Human annotators assess quality, safety, and usefulness of outputs. Platforms include **Chatbot Arena, Argilla, and Labelbox**.

5. *How do **Temperature**, **Top-K**, and **Top-P** sampling parameters influence the creativity and coherence of model outputs?*

Temperature, Top-K, and Top-P are sampling hyperparameters that control how an LLM chooses the next word, balancing **coherence vs. creativity**.

- **Temperature** controls randomness by scaling probabilities.
  - **Low (≈0.2):** very deterministic and focused.
  - **Medium (≈0.7):** balanced creativity.
  - **High (≈1.2):** more diverse but can become incoherent.
- **Top-K** limits the model to the **K most probable tokens**, ignoring the rest. This keeps outputs coherent while allowing some randomness among top choices.
- **Top-P (Nucleus Sampling)** selects from the **smallest set of tokens whose cumulative probability exceeds P** (e.g., 0.96). The candidate pool changes dynamically—smaller when the model is confident, larger when it's uncertain.

6. *What are the most effective strategies for optimizing LLM performance (e.g., quantization)?*

Optimizing **Large Language Models (LLMs)** involves four main strategies:

1. **Quantization (Compression)** – Reduces model size by lowering weight precision (e.g., FP32 → INT8). This saves memory and speeds up inference. **QLoRA** further compresses models to 4-bit for efficient fine-tuning.
2. **Fine-Tuning (Parameter Optimization)** – Adapts models to specific tasks.
   - **Full Fine-Tuning (SFT):** updates all parameters (expensive).
   - **PEFT:** updates only small parts of the model (e.g., **Adapters, Prompt Tuning, LoRA**).
   - **RLHF:** improves outputs using human feedback.
3. **Prompt Engineering** – Improves performance without changing model weights by designing better inputs (e.g., **Zero/Few-shot prompting, Chain-of-Thought, RAG**).
4. **Hyperparameter Optimization** – Adjusts generation behavior using settings like **Temperature, Top-K, Top-P**, and decoding strategies such as **Greedy decoding** or **Beam Search**.

7. *What is **Prompt Engineering**, and what are the psychological or logical frameworks (like Chain-of-Thought) that make it work?*

Prompt Engineering is the practice of designing input prompts to guide an LLM toward desired outputs. It "programs" the model using natural language and **does not change the model's parameters**, unlike fine-tuning.

It works through **In-Context Learning**, where examples or instructions in the prompt help the model adapt its response to the given context.
Key techniques include:

- **Chain-of-Thought (CoT):** encourages step-by-step reasoning before giving the final answer.
- **Zero-shot prompting:** no examples; relies on the model's pretraining.
- **Few-shot prompting:** provides a few examples to guide the output.

Other methods include **Generated Knowledge Prompting, Self-Consistency, Retrieval Augmented Generation (RAG), and ReAct prompting**.

8. *Explain the **PEFT (Parameter-Efficient Fine-Tuning)** ecosystem and its benefits over full fine-tuning.*

Parameter-Efficient Fine-Tuning (PEFT) improves a pre-trained model for specific tasks by **training only a small subset of parameters** while keeping the original model weights frozen. This makes it **much cheaper and faster** than full fine-tuning, reducing memory, training time, and storage while achieving similar performance.
PEFT methods fall into three categories:

1. **Additive Methods** – Add small trainable components without changing original weights (e.g., **Adapters**, **Soft Prompts / Prompt Tuning**).
2. **Selective Methods** – Fine-tune only specific parts of the model (e.g., **top layers**, **bias parameters**, or **sparse parameter subsets**).
3. **Reparameterization Methods** – Use mathematical shortcuts to represent weight updates efficiently (e.g., **LoRA**, **Intrinsic SAID**).

9. *What are the ethical and technical challenges (e.g., hallucination, catastrophic forgetting) when fine-tuning LLMs?*

The course slides **do not explicitly define** hallucination, catastrophic forgetting, or ethical challenges, but they discuss related issues such as **bias, safety, and computational cost**.

- **Safety and Bias:** Human evaluation is important because humans judge helpfulness, safety, and nuance. LLM-as-a-Judge systems can also introduce bias.
- **Compute Challenges:** Full fine-tuning is extremely expensive and memory-intensive, which motivates **PEFT methods like LoRA**.

Common ML definitions (outside the slides):

- **Catastrophic Forgetting:** When a model fine-tuned on new data loses previously learned knowledge.
- **Hallucination:** When an LLM confidently generates incorrect or fabricated information.
- **Ethical Challenges:** Bias amplification, harmful content, privacy risks, and high energy/compute usage.

10. MLM CLM RAG Knowledge Graph

- **MLM (Masked Language Modeling):** A self-supervised pre-training objective where a certain percentage of input tokens (typically around 15%) are randomly hidden or "masked". The model is then trained to predict these original masked words by analyzing the surrounding context from both the left and the right. It is frequently compared to a "cloze test," an exercise where a participant fills in the blank portions of a text.

- **CLM (Causal Language Modeling):** A training objective where the model is tasked simply with guessing the next word in a sentence. For example, given the preceding words "My name is", the model learns to sequentially predict the next logical word, such as "Sylvain".

- **RAG (Retrieval Augmented Generation):** An advanced prompt engineering technique and system architecture that enhances an LLM's outputs by connecting it to an external database (such as ChromaDB). When a user asks a question, a "Smart Retriever" looks up relevant documents from this specific knowledge base, and the LLM acts as a generator to produce an "explicit source-informed answer" based on that retrieved data.

- **Knowledge Graph:** Described in the course as a "flexible data structure" consisting of a set of facts or nodes. The materials note that a set of Knowledge Graph facts often does not form a fully connected graph, and they are utilized in graph-based machine learning tasks where Transformers and Graph Neural Networks (GNNs) learn their structural connection patterns.